現代與後量子密碼學導論

Introduction to Modern and Post-quantum Cryptography

Fall 2026

Part 1: Provable Security

1.7: Hard Problems & Diffie-Hellman

Zi-Yuan Liu

CC BY NC SA

The original slides were prepared by Nadim Kobeissi.

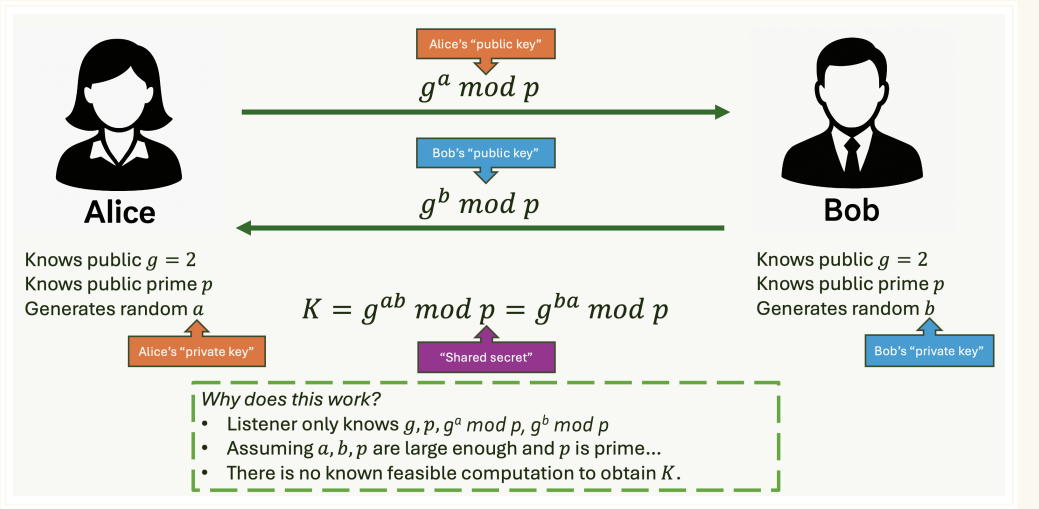
Section 1

Hard Problems

Hard problems

- Cryptography is largely about equating the security of a system to the difficulty of solving a math problem that is thought to be computationally very expensive.
- With cryptography, we get security systems that we can literally mathematically prove as secure (under assumptions).
- Also, this allows for actual magic.
  - Alice and Bob meet for the first time in the same room as you.
  - You are listening to everything they are saying.
  - Can they exchange a secret without you learning it?

Time for actual magic



No known feasible computation

- The discrete logarithm problem:
  - Given a finite cyclic group G, a generator $g \in G$, and an element $h \in G$, find the integer x such that $g^x = h$
- In more concrete terms:
  - Let p be a large prime and let g be a generator of the multiplicative group $\mathbb{Z}_p^*$ (all nonzero integers modulo p).
  - Given:
    - $g \in \mathbb{Z}_p^*, h \in \mathbb{Z}_p^*$
    - Find $x \in \{0, 1, \ldots, p-2\}$ such that $g^x \equiv h \pmod{p}$
  - This problem is believed to be computationally hard when p is large and g is a primitive root modulo p.
    - "Believed to be" = we don't know of any way to do it that doesn't take forever, unless we have a strong, stable quantum computer (Shor's algorithm)

Hard problems

Asymmetric Primitives

- Diffie-Hellman, RSA, ML-KEM, etc.
- “Asymmetric” because there is a “public key” and a “private key” for each party.
- Algebraic, assume the hardness of mathematical problems (as seen just now).

Symmetric Primitives

- AES, SHA-2, ChaCha20, HMAC...
- “Symmetric” because there is one secret key.
- Not algebraic but unstructured, relying on their understood resistance to n years of cryptanalysis.
- Can act as substitutes for assumptions in security proofs!
  - Example: hash function assumed to be a “random oracle”

Categorizing computational hardness

Easy Problems

- Solvable in polynomial time.
- **Examples**: Sorting, searching.
- Running time: $\mathcal{O}(n^c)$ for some constant c.
- Generally scales reasonably with input size.
- Class P (Polynomial time).

Hard Problems

- No known polynomial-time solution.
- **Example**: Factoring the product of two large primes.
- Best known running time: exponential or subexponential in the input size.
- Becomes impractical quickly as input grows.

Why hard problems matter

- Hard problems **create asymmetry between legitimate users and attackers**: easy in one direction, hard in reverse.
  - Easy to multiply two large primes; hard to factor the product.
  - Easy to compute $g^x \bmod p$; hard to recover x.
- Public-key cryptography is built on exactly these two problems:
  - **RSA**: integer factorization.
  - **Diffie-Hellman**: discrete logarithm.
- The security of these schemes depends on the **continued** hardness of these problems: an algorithmic breakthrough breaks the crypto, no bug required.

Section 2

Diffie-Hellman

The key exchange problem

- Alice and Bob want to communicate securely over the internet.
- They've never met before and share no secrets.
- How can they establish a shared secret key for encryption?
- Traditional approach: meet in person, exchange keys physically.
- **Problem**: This doesn't scale for the internet!

The Challenge

Create a shared secret between two parties who have never communicated before, even when an eavesdropper can see everything they send to each other.

The magic of Diffie-Hellman

- Whitfield Diffie and Martin Hellman solved this “impossible” problem.
- Their solution (1976) came one year **before** RSA (1977).
  - Hellman himself says it should be called *Diffie-Hellman-Merkle*: Ralph Merkle had the idea of public-key distribution first.
  - GCHQ’s Malcolm Williamson had found the same scheme in 1974; it stayed classified until 1997.
- Uses the **discrete logarithm problem** as its foundation.
- Allows two strangers to create a shared secret in public!

This was the birth of modern cryptography

What makes the discrete logarithm hard?

Take $p = 17$ and $g = 3$ (a generator of $\mathbb{Z}_{17}^*$). Easy one way, hard the other:

Easy direction: computing powers

- $3^1 \bmod 17 = 3$
- $3^2 \bmod 17 = 9$
- $3^3 \bmod 17 = 27 \bmod 17 = 10$
- $3^4 \bmod 17 = 81 \bmod 17 = 13$
- $3^5 \bmod 17 = 243 \bmod 17 = 5$

Hard direction: finding the exponent

- Given $g = 3$, $p = 17$ and the result 5: which exponent was it?
- With small numbers: easy, try them all (answer: $x = 5$, the only one below 16).
- With a 2048-bit p: practically impossible, even with every computer on Earth.

Computing $g^x \bmod p$ takes about $\log_2 x$ multiplications (square-and-multiply); going back has no known shortcut.

Mathematical groups: the foundation

What is a Mathematical Group?

A set of elements with an operation that follows specific rules.

- Think of it as a **mathematical playground** with consistent rules.
- For cryptography, we use $\mathbb{Z}_p^*$: numbers $\{1, 2, 3, \ldots, p-1\}$ with multiplication mod p.

- **Example**: $\mathbb{Z}_5^* = \{1, 2, 3, 4\}$ with multiplication mod 5
  - $3 \times 4 = 12 \equiv 2 \pmod 5$
  - $2 \times 3 = 6 \equiv 1 \pmod 5$
- The “rules” ensure the math works consistently for cryptography.

Group rules (simplified)

For our cryptographic group $\mathbb{Z}_p^*$, these rules always hold:

- **Closure**: Multiplying any two elements gives another element in the group
  - In $\mathbb{Z}_5^*$: $2 \times 3 = 1$ (still in the group!)
- **Identity**: There's a special element (1) that doesn't change others
  - $1 \times 4 = 4$, $1 \times 2 = 2$, etc.
- **Inverses**: Every element has a "partner" that multiplies to 1
  - In $\mathbb{Z}_5^*$: $2 \times 3 = 1$, so 2 and 3 are inverses
- **Associativity**: $(a \times b) \times c = a \times (b \times c)$

Why care? These rules guarantee that our cryptographic operations will behave predictably!

Generators: reaching every element in the group

Generator

An element g whose powers $g^1, g^2, g^3, \ldots$ produce **every element in the group**.

- In $\mathbb{Z}_5^*$, let's try $g = 2$:
  - $2^1 \bmod 5 = 2$
  - $2^2 \bmod 5 = 4$
  - $2^3 \bmod 5 = 3$
  - $2^4 \bmod 5 = 1$
- We got $\{2, 4, 3, 1\}$: that's all the elements! So $g = 2$ is a generator.
- **Generators are crucial**: They let us express every group element as a power of g.

The discrete logarithm problem (DLP)

Discrete Logarithm Problem

Given g, p, and $h = g^x \bmod p$, find the secret exponent x.

- **"Discrete"** because we work with integers, not real numbers.
- **"Logarithm"** because we're finding the exponent (like $\log_2(8) = 3$).
- **Example**: Given $g = 3$, $p = 17$, $h = 10$, find x such that $3^x \equiv 10 \pmod{17}$.
  - Answer: $x = 3$ (since $3^3 = 27 \equiv 10$)
  - Easy with small numbers, hard with large ones!
- For cryptographic-sized numbers (2048+ bits), no efficient algorithm is known.

DLP vs. factoring: similarly hard

Factoring Problem

- Given $N = p \times q$, find p and q
- Used in RSA (1977)
- Well-known, intuitive

Discrete Logarithm

- Given $g^x \bmod p$, find x
- Used in Diffie-Hellman (1976)
- Less intuitive, more mathematical

- **Security equivalence**: n-bit factoring $\approx$ n-bit discrete logarithm
- Both are vulnerable to Shor's quantum algorithm
- Both are **not** known to be NP-hard
- Algorithms for both problems share similar techniques

Attacking the discrete logarithm: can we beat brute force?

- **Brute force**: try every $x \in \{0, 1, \dots, p-2\}$ until $g^x \equiv h \pmod{p}$.
  - Running time: $\mathcal{O}(p)$; for a 2048-bit prime, longer than the age of the universe.
- But brute force is the *dumbest* possible algorithm. Surely we can be smarter?
- Two classic **generic** algorithms beat brute force in **any** group:
  - **Baby-step giant-step**: trade memory for time.
  - **Pollard's rho**: same speed, (almost) no memory.
- Both bring the running time down from $\mathcal{O}(p)$ to $\mathcal{O}(\sqrt{p})$.

Spoiler

$\sqrt{p}$ is a colossal improvement over p, but for a 2048-bit prime, $\sqrt{p} \approx 2^{1024}$. Still hopeless.

Diffie-Hellman: the mathematical version

Setup: Alice and Bob agree on public values g (generator) and p (large prime)

1. **Alice**: Chooses secret a, computes $A = g^a \bmod p$, sends A to Bob
2. **Bob**: Chooses secret b, computes $B = g^b \bmod p$, sends B to Alice
3. **Alice**: Computes shared secret $S = B^a \bmod p = (g^b)^a \bmod p = g^{ab} \bmod p$
4. **Bob**: Computes shared secret $S = A^b \bmod p = (g^a)^b \bmod p = g^{ab} \bmod p$

Result: Alice and Bob both have $S = g^{ab} \bmod p$ without ever sharing a or b!

Diffie-Hellman example with small numbers

Setup

- Public parameters: $p = 13, g = 2$
- Alice picks secret: $a = 5$
- Bob picks secret: $b = 7$

Public Exchange

- Alice computes: $A = g^a \bmod p = 6$
- Bob computes: $B = g^b \bmod p = 11$
- Alice sends $A = 6$ to Bob
- Bob sends $B = 11$ to Alice

Shared Secret Computation

- Alice computes:
  $S = B^a \bmod p = 11^5 \bmod 13$
  - $= 161051 \bmod 13 = 7$
- Bob computes:
  $S = A^b \bmod p = 6^7 \bmod 13$
  - $= 279936 \bmod 13 = 7$
- **Shared secret:** $S = 7$

Eavesdropper sees only:

- $p = 13, g = 2, A = 6, B = 11$
- (In the real world, p, a, and b are much larger numbers)

Your turn: run Diffie-Hellman by hand

Public parameters: $p = 23, g = 5$. Alice picks $a = 6$, Bob picks $b = 15$. Work in pairs and reduce mod 23 after every multiplication.

1. What does Alice send? ($A = 5^6 \bmod 23$)
2. What does Bob send? ($B = 5^{15} \bmod 23$)
3. What shared secret do they both end up with?
4. You are Eve and you see p, g, A, B. How many exponents might you have to try to find a?

Answers

$A = 8$ (since $5^2 = 25 \equiv 2$, so $5^6 = 2^3 = 8$); $B = 19$; $S = B^6 \bmod 23 = A^{15} \bmod 23 = 2$.
Eve: at most 22 tries here; 2^{2048} for a 2048-bit prime, still 2^{1024} with baby-step giant-step.
And Eve doesn't actually need a or b, only S: that is the next slide's problem.

The computational Diffie-Hellman (CDH) problem

Computational Diffie-Hellman (CDH) Problem

Given $g^a \bmod p$ and $g^b \bmod p$, compute the shared secret $g^{ab} \bmod p$ without knowing the secret values a and b.

- **Motivation**: Even if an eavesdropper captures the public values g^a and g^b, they shouldn't be able to determine the shared secret g^{ab}.
- **Example**: Given $A = 6$ and $B = 11$ from our earlier example, can you compute $S = 7$?
  - With $p = 13$ you can brute-force it in seconds; with a 2048-bit p, nobody knows how.
- **Real-world relevance**: This is exactly what an attacker faces when trying to break Diffie-Hellman.

CDH vs. DLP: the relationship

- **Key insight**: If you can solve DLP, then you can also solve CDH.
  - Given g^a and g^b, use DLP to find a and b
  - Then compute g^{ab} directly
- **Mathematical relationship**: DLP is **at least as hard** as CDH.
  - CDH $\leq$ DLP (CDH reduces to DLP)
- **Open question**: Is CDH at least as hard as DLP?
  - We don't know if solving CDH allows you to solve DLP!
  - Maybe there's a clever way to compute g^{ab} without finding a and b
- **Security assumption**: We assume CDH is hard even if it turns out to be easier than DLP.

The decisional Diffie-Hellman (DDH) problem

Decisional Diffie-Hellman (DDH) Problem

Given $g^a \bmod p$, $g^b \bmod p$, and a value X that is either $g^{ab} \bmod p$ (the real shared secret) or $g^c \bmod p$ for a random c, each with probability $1/2$: determine which one X is.

- **Why do we need this?** Indistinguishability!
  - What if an attacker can compute the first 32 bits of g^{ab}?
  - CDH isn't broken, but the attacker learns something. That partial leak may be all an application needs to fail.
- **DDH ensures**: The shared secret g^{ab} is **indistinguishable** from a random group element.
- **Where it's used**: security proofs of ElGamal, Cramer-Shoup and many key exchange protocols. In our library notation, it's what lets us swap the real key for a uniform one.

DDH vs. CDH: the hierarchy

- **Key relationship**: If you can solve CDH, then you can solve DDH.
  - Given (g^a, g^b, X), use CDH to compute g^{ab}
  - Check if $X = g^{ab}$; if yes, then X is the real shared secret
- **Hardness hierarchy**: DDH $\leq$ CDH $\leq$ DLP.
  - DDH is **no harder** than CDH.
  - CDH is, in turn, no harder than DLP.
- **Surprising fact**: DDH is **not hard** in certain groups!
  - In $\mathbb{Z}_p^*$, DDH can be broken using the Legendre symbol (quadratic residuosity): g^{ab} is a square exactly when g^a or g^b is, while a random g^c is a square only half the time.
  - But CDH remains hard in the same group.
- **Solution**: work in a group of large **prime order** q, where this leak disappears: the order-q subgroup of $\mathbb{Z}_p^*$ for $p = 2q + 1$, or a prime-order elliptic curve. There DDH is believed hard.

Section 2

Diffie-Hellman

Subsection 2.1

Real-World Diffie-Hellman

Real-world Diffie-Hellman

- **TLS/HTTPS**: Your browser uses Diffie-Hellman to establish secure connections.
- **Signal**: Uses elliptic-curve Diffie-Hellman for key exchange.
- **SSH**: Secure shell connections use Diffie-Hellman for key agreement.
- **VPNs**: Many VPN protocols rely on Diffie-Hellman for establishing tunnels.

Modern Diffie-Hellman Variants

- **Elliptic Curve Diffie-Hellman (ECDH)**: Same idea, different mathematical group.
- **Post-quantum alternatives**: New key exchange methods for the quantum era.

More on both of the above in future course topics!

Diffie-Hellman key exchange in practice

How does this actually work in the real world?

1. **Parameter generation**: Choose secure values for p and g
   - p must be a large prime (2048+ bits)
   - g must be a generator of a large subgroup
2. **Key generation**: Each party picks a random secret
   - Alice picks a randomly from $\{1, 2, \dots, p-2\}$
   - Bob picks b randomly from $\{1, 2, \dots, p-2\}$
3. **Public key computation**: Each party computes their public value
4. **Key exchange**: Public values are sent over the network
5. **Shared secret derivation**: Each party computes the final shared secret

Don’t use g^{ab} directly as a key

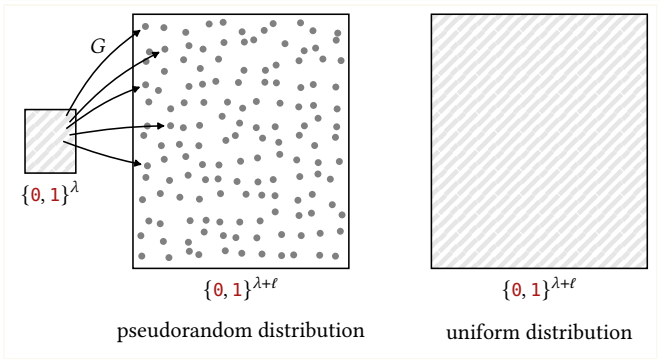
- Alice and Bob share $S = g^{ab} \bmod p$. **Tempting**: just use S as an AES key!
- **Problem**: S is **not** a uniformly random bitstring; it’s a specific element of a specific algebraic group.
  - It always lies in the subgroup generated by g, and depending on g may always be a **quadratic residue**.
  - Its byte-length can vary (leading zero bytes), a classic source of real bugs: the **Raccoon attack** (2020) on TLS 1.2 exploited exactly this.
- Structured, biased input makes a **terrible** symmetric key: it hands an attacker a head start.

Rule of thumb

Never feed a shared secret straight into a cipher; always pass it through a **KDF** first.

Pseudorandom and uniform distributions

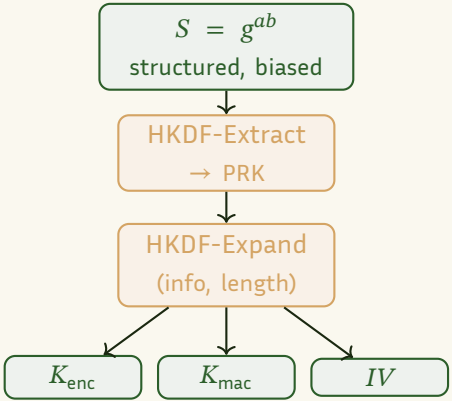
Reminder



Source: The Joy of Cryptography

Whitening the secret: key derivation functions

- A **KDF** takes structured, possibly biased input and outputs **uniformly random**, independent key material.
- The standard choice is **HKDF**, used in TLS 1.3, Signal, Noise, and more:
  - **Extract**: $\mathrm{PRK} = \mathrm{HMAC}(salt, S)$.
  - **Expand**: stretch PRK with a context string `info` into as much key material as needed.
- One shared secret → many **independent** keys: encryption key, MAC key, IV, …



TLS handshake: Diffie-Hellman in action

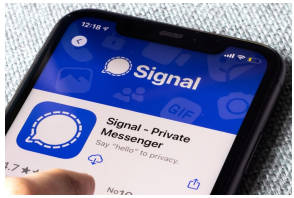
When you visit https://gmail.com, here's what happens (TLS 1.2 flavour, simplified):

1. **Client Hello**: Your browser says "I want to talk securely"
2. **Server Hello**: Gmail's server responds with its certificate and its DH share
   - Includes p, g, and the server's public value $g^b \bmod p$
3. **Client Key Exchange**: Your browser generates its own secret a and sends $g^a \bmod p$
4. **Secret computation**: Both sides compute $g^{ab} \bmod p$
5. **Key derivation**: The shared secret goes through a KDF to produce the session keys
6. **Secure communication**: All further messages are encrypted with these keys

Result: Your password is encrypted before leaving your computer! (TLS 1.3 sends the browser's share in its first message and uses named groups such as X25519: topic 2.1.)

Signal's double ratchet: DH everywhere

- **Initial key exchange**: X3DH (Extended Triple DH)
  - Combines **three** DH key exchanges; since 2023, PQXDH adds a post-quantum KEM.
  - Works even when recipient is offline ("*asynchronous*" protocol).[a]
- **Ongoing communication**: Uses Double Ratchet
  - New DH key exchange with every reply!
  - Provides "forward secrecy" and "post-compromise security".
  - Phone compromised today? Yesterday's messages stay secure, and once you recover, tomorrow's are secure again.



Signal performs dozens or even hundreds of DH key exchanges per conversation.

[a] Everything on this slide will be covered in much more detail later in the course.

Section 2

Diffie-Hellman

Subsection 2.2

Attacks on Diffie-Hellman

The dark side: unauthenticated Diffie-Hellman

Even with fully correct Diffie-Hellman implementations, there's a serious problem...

- **The vulnerability**: Basic DH has no authentication
  - Alice can't verify she's talking to Bob
  - Bob can't verify he's talking to Alice
- **The attack**: Man-in-the-middle (MITM)
  - Mallory sits between Alice and Bob
  - Alice does DH with Mallory, thinking it's Bob
  - Bob does DH with Mallory, thinking it's Alice
  - Mallory can read and modify everything!
- **Real-world impact**: This attack is practical and devastating!

Man-in-the-middle attack on DH

How Mallory breaks “secure” communication:

1. **Alice → Mallory**: Alice sends g^a (thinking it goes to Bob)
2. **Mallory → Bob**: Mallory sends g^m (Bob thinks it’s from Alice)
3. **Bob → Mallory**: Bob sends g^b (thinking it goes to Alice)
4. **Mallory → Alice**: Mallory sends g^m (Alice thinks it’s from Bob)
5. **Result**:
   - Alice and Mallory share secret g^{am}
   - Bob and Mallory share secret g^{bm}
   - Alice and Bob don’t share any secret!
6. **Communication**: Alice encrypts with g^{am}, Mallory decrypts, reads/modifies, re-encrypts with g^{bm} for Bob

Alice and Bob never know they’ve been compromised!

Why MITM attacks succeed

- **Public values look random**: g^a and g^m are indistinguishable.
  - Both appear to be random group elements.
  - No way to tell if they come from the intended party.
- **No identity verification**: DH only establishes a shared secret.
  - Doesn't prove who you're sharing it with!
  - Like agreeing on a secret handshake with someone wearing a mask.
- **Active vs. passive attacks**:
  - DH protects against **passive** eavesdropping.
  - Does nothing against **active** manipulation.

Solution: authenticated key exchange

Authenticated Key Exchange (AKE)

Key exchange that verifies the identity of the parties involved, preventing man-in-the-middle attacks.

- **Core idea**: Combine DH with authentication mechanisms.
- **Common approaches**:
  - **Digital signatures**: Sign the DH public values (TLS).
  - **Pre-shared keys**: Use existing shared secrets (IPsec).
  - **Certificates**: Use a trusted third party (Certificate Authority in HTTPS).
  - **Password-based**: Derive authentication from passwords (PAKE protocols like SRP).
- **Goal**: Ensure that Alice and Bob can verify they're really talking to each other.

TLS: authenticated DH with certificates

How HTTPS prevents MITM attacks:

1. **Server authentication**: Gmail sends its certificate and a **signature** on g^b
   - The certificate binds the signing key to "gmail.com"; the signature binds g^b to that key
   - The certificate is signed by a trusted Certificate Authority (CA)
2. **Certificate verification**: Your browser checks:
   - Is the signature valid?
   - Is the CA trusted?
   - Does the certificate match "gmail.com"?
   - Has the certificate expired?
3. **If verification passes**: You know you're really talking to Gmail
4. **If verification fails**: Browser shows scary warnings!

Result: MITM attacks become much harder (but not impossible!)

Signal: authenticated DH with fingerprints

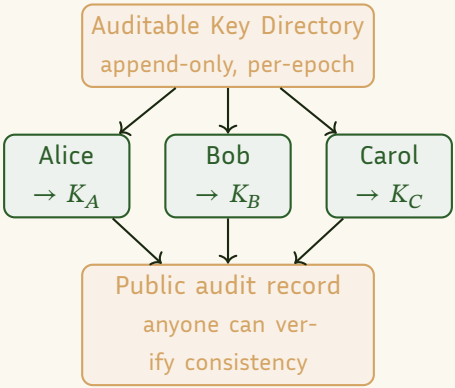
- **The bootstrapping problem**: How do Alice and Bob initially authenticate?
  - No pre-existing certificates.
  - No trusted third parties.
- **Signal's solution**: Security numbers (fingerprints).
  - Each conversation gets a unique 60-digit number.
  - Derived from both parties' long-term identity keys.
- **Manual verification**: Users compare numbers out-of-band.
  - Read over the phone...
  - Show in person...
  - Send via different app...



Signal security number verification screen.

WhatsApp: key transparency

- **The problem**: Manual QR verification scales terribly: a group of 100 needs $\binom{100}{2} = 4{,}950$ pairwise checks!
- **WhatsApp (2023)**: **Key transparency** verifies connections automatically, no user action.[a] Signal followed in August 2026 (topic 2.5).
- **How**: Server keeps an **append-only key directory**; a public **audit record** proves it is correct.



Builds on CONIKS/SEEMless/Parakeet; open-sourced as a Rust **AKD** crate.

[a] https://appliedcryptography.page/paper/#whatsapp-transparency

SSH: authenticated DH with host keys

How SSH prevents server impersonation:

- **First connection**: Server presents its “host key” along with its DH public value
  - SSH shows you a fingerprint: `SHA256:ABC123...`
  - You’re supposed to verify this out-of-band (but nobody does!)
- **Trust on first use (TOFU)**: Client remembers the host key
  - Stored in `~/.ssh/known_hosts`
- **Subsequent connections**: Client checks if the host key matches
  - If different, SSH gives you a heart attack: `WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!`
  - If same: Connection proceeds normally
- **User authentication**: Usually with passwords or public keys

Weakness: TOFU is vulnerable on the very first connection!

Section 3

RSA

RSA: the other public-key pioneer

One year after Diffie-Hellman came RSA (1977)

- **Named after**: Rivest, Shamir, and Adleman (MIT researchers)
- **Different approach**: Based on the hardness of factoring large numbers
- **Key innovation**: One trapdoor gives both public-key encryption AND digital signatures
- **Secret history**: GCHQ's Clifford Cocks found the same scheme in 1973 (classified until 1997)

RSA's Core Idea

Anyone can encrypt messages to you using your public key, but only you can decrypt them with your private key.

How RSA works (simplified)

The mathematical foundation:

1. **Key generation**:
   - Choose two large primes p and q
   - Compute $n = p \times q$ (public modulus)
   - Choose public exponent e (commonly 65537)
   - Compute private exponent d such that $e \times d \equiv 1 \pmod{\phi(n)}$
     - $\phi(n)$ is Euler's totient function: counts integers less than n that are coprime to n
     - For $n = p \times q$: $\phi(n) = (p-1)(q-1)$
2. **Encryption**: $c = m^e \bmod n$ (anyone can do this)
3. **Decryption**: $m = c^d \bmod n$ (only the private key holder can do this)

Security assumption: Factoring n back into p and q is computationally infeasible.

RSA's dual functionality

Encryption

- Public key encrypts
- Private key decrypts
- Provides confidentiality
- Anyone can send you secrets

Digital Signatures

- Private key signs
- Public key verifies
- Provides authenticity
- You can prove you wrote something

- **Historical significance**: First algorithm to do both!
- **Widespread adoption**: Used in SSL/TLS, email encryption, code signing
- **Patent controversy**: Was patented until 2000, limiting early adoption

RSA in practice

Where RSA dominated for decades:

- **HTTPS certificates**: RSA signatures dominated for two decades; ECDSA certificates only became common in the late 2010s
- **Email encryption**: PGP/GPG relied heavily on RSA
- **Software signing**: Windows, macOS, Linux packages
- **SSH keys**: RSA was the default for years

Key sizes and security:

- 1024-bit RSA: NIST disallowed it after 2013 (about 80-bit security)
- 2048-bit RSA: Current minimum (about 112-bit security), but showing its age
- 4096-bit RSA: Secure but slow and unwieldy
- Compare to ECDSA: 256-bit keys offer similar security to 3072-bit RSA!

Why we're not covering RSA in depth

- **On its way out**: Being actively replaced everywhere
  - TLS 1.3, SSH, Signal and WireGuard all default to elliptic curves
  - Trail of Bits, 2019: "Seriously, stop using RSA"
- **Its math doesn't travel**: DH moves to elliptic curves unchanged; RSA's trapdoor is tied to factoring, where the number field sieve keeps pushing key sizes up
- **Performance**: Terribly slow to sign or decrypt
  - 4096-bit RSA signature: ~10ms; Ed25519 signature: ~0.1ms
  - (RSA *verification* is fast, which is why it lingered in certificates)
- **Implementation pitfalls**: Textbook RSA is deterministic and malleable, so it needs padding, and PKCS#1 v1.5 padding gave us Bleichenbacher's oracle attack (1998), rediscovered as ROBOT in 2018
- **The future is elliptic curves and post-quantum**: Time better spent there!

現代與後量子密碼學導論

Introduction to Modern and Post-quantum Cryptography

Fall 2026

Part 1: Provable Security

1.7: Hard Problems & Diffie-Hellman

Zi-Yuan Liu

CC BY NC SA

The original slides were prepared by Nadim Kobeissi.